

Maestría en Economía
Facultad de Ciencias Económicas -- Universidad Nacional de La Plata

Seminario de Stata

clase 2

Santiago Garganta
(CEDLAS-UNLP)

Universidad Nacional de La Plata
2017

Clase anterior

- Introducción al programa
- Explicaciones generales (recomendaciones, sintaxis general, modo de trabajar en stata, ponderadores, etc.)
- Análisis de algunos comandos básicos (summarize, generate, replace, sort, preserve y restore, by, drop, table, tabulate y algunas funciones del comando egen)

Clase 2

- Algunas aclaraciones de los comandos analizados en la clase anterior.
- Finalizar con los comandos básicos (con la encuesta de hogares de Ecuador 2011) y resolver ejercicios iniciales.
- Organización del trabajo en archivos .do y .log
- Tipos de datos
- Gestión de base de datos

egen

- `gen suma1 = sum(pondera)`
- `egen suma2 = sum(pondera)`
- `egen suma3 = total(pondera)`

egen

- `gen suma1 = sum(pondera)`
- ~~`egen suma2 = sum(pondera)`~~
- `egen suma3 = total(pondera)`

egen (más opciones)

- `gen aux = ila + inla`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- `egen aux1 = rsum(ila inla)`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- ~~`egen aux1 = rsum(ila inla)`~~
- ~~`egen aux2 = rsum(ila inla), missing`~~
- `summ aux*`
- `drop aux*`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- `egen aux1 = rowtotal(ila inla)`
- `egen aux2 = rowtotal(ila inla), missing`
- `summ aux*`
- `drop aux*`

La Utilización de Ponderadores

- El Stata permite distintos tipos de ponderación (help weight):
 - **fweights = frequency** (indican el número de observaciones duplicadas)
 - **pweights = sampling** (representan la inversa de la probabilidad de que la observación sea incluida de acuerdo al diseño de la muestra)
 - **awweights = analytic** (las observaciones representan promedios y los ponderadores son inversamente proporcionales a la varianza del mismo)
 - **iweights = importance** (indican la importancia de cada observación de acuerdo a algún criterio)
- En general, los comandos de Stata tienen asignado un tipo de ponderación por defecto; es el que utilizaremos.

La Utilización de Ponderadores

- **Sin ponderar**

summarize ipcf

- **Ponderado**

summarize ipcf [w=pondera]

Otros Comandos

- **display:** muestra operaciones o texto que sigue al comando
- `display "hola"`
- `display 2 * 2`
- `display ipcf[1]`
- `display ipcf[_N]`
- `display "Tasa de desempleo = " 2*4`

Ejercicio 1

- Calcular la cantidad de miembros por hogar promedio en Ecuador 2011

summarize miembros

???????

Ejercicio 1

- Calcular la cantidad de miembros por hogar promedio en Ecuador 2011

summarize miembros [w=pondera]

Por qué no?

Ejercicio 1

- Calcular la cantidad de miembros por hogar promedio en Ecuador 2011

sort id

egen hogar=tag(id)

summarize miembros [w=pondera] if hogar==1

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

gen a = 1

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

gen a = 1

bysort id: egen miembros1=total(a)

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

gen a = 1

bysort id: egen miembros1=total(a)

compare miembros miembros1

Organización de un proyecto

- Hasta el momento solo hemos trabajado con la ventana de comandos
- Archivos **.do**: archivos de texto que contienen toda la secuencia de comandos
- Los resultados que surgen de aplicar dicha secuencia los podemos almacenar en un archivo **.log**

Archivos do

- Los “do” son archivos de texto plano con código de Stata que se ejecuta de forma secuencial, línea por línea – programación
 - pueden editarse con el bloc de notas de Windows
 - no recomendamos el uso del editor de archivos do de Stata – al menos, hasta la versión 11
- Es recomendable trabajar siempre con archivos do – permiten replicar resultados fácilmente.

Archivos do

- Para los archivos .do existe un editor de Stata y también editores de texto externos (editplus, crimson, notepad++)
- Los editores externos contienen algunas funciones adicionales al de Stata.
- Utilizaremos el notepad++

Archivos do

- Ver intro_stata.do.
- Correr el .do
- Comentarios?

Ejercicio inicial (.do)

- Trabajar en la carpeta
“c:\Seminario_Stata\clase2”
- En esa carpeta está la base de datos
eph2010_resumida.dta
- Crear un archivo .do
- Para ello abrir el notepad++
- Guardar el archivo .do con el nombre
ejercicio1.do
- Recuerden agregarle la extensión .do cuando
guardan el archivo

Ejercicio inicial

- En este primer archivo .do:
 - Ubicarnos en la carpeta donde vamos a trabajar
 - Abrir la base de datos
 - Mostrar el contenido de la base de datos
 - Obtener la edad promedio de la muestra
 - Cerrar
- La mecánica es correr el do por primera vez y luego para los cambios posteriores utilizar la ventana Review

Ejercicio inicial

- Almacenar los resultados en un archivo **.log**
- Es importante saber que existe:
 - Pero vamos a estar almacenando los resultados que nos interesan (matrices, tablas, exportacion de resultados a archivos .xls)
 - Últimas clases
- En el archivo .log se guardan los resultados entre el comando **log using** y **log close**

Ejercicio inicial

- En el archivo .log se guardan los resultados entre el comando **log using** y **log close**
- El comando log using resulta en un error si ya existe un archivo log en uso o abierto
- Por eso debemos cerrar cualquier log que esté en uso (log close)
- El problema es que log close dará un error si no hay un log abierto para cerrar
- Introducción del comando capture (OJO!)

Base de datos

- La base de datos se corresponde con la Encuesta Permanente de Hogares 2010 (EPH), resumida
- Datos socioeconómicos, laborales e ingresos de los hogares.
- La base de datos es una tabla de doble entrada:
 - Las columnas son variables
 - Las filas son observaciones
 - En cada celda se pueden guardar diferentes tipos de datos (numéricos, palabras, fechas)

Datos

- Comando describe muestra el tipo de datos.
- Numérico, hay varios tipos
 - byte, integer, Long (enteros). Se diferencian por los límites
 - Float, Double (con decimales). Se diferencian no solo por la extensión sino por la cantidad de decimales

Datos

- Cuando genero una variable puedo aclarar el tipo de dato que quiero que se genere. Por ejemplo: **generate byte prueba = 0**
- Pero Stata cambia el tipo de dato de la variable numérica en función de los datos que se ingresen en la misma
- **replace prueba = 150**
- **replace prueba = 2.5**

Datos

- Stata trabaja con el punto como separador de decimal y coma separador de miles
- Asegurarse que la configuración regional de la pc se encuentre en línea con Stata para evitar inconvenientes
- Inicio | Panel de control | Configuración regional

Datos

- Variables String
 - generate str auxiliar = ""
 - replace auxiliar = "hola"
 - replace auxiliar = "8"
- Para transformar una variable string en numérica puedo aplicar el comando encode:
 - encode pais, gen(npais)
 - **npais** será una variable numérica con distintos valores según cada una de las palabras presentes en la variable **pais**

Datos

- Para transformar una variable string (con números) en numérica puedo aplicar el comando destring:

destring auxiliar, replace

- La variable region es numerica o string?

Datos

- Puedo transformar una variable numérica en string:
 - **generate reg_str = string(region)**
- Puedo transformar una variable string en numérica de la siguiente manera (alternativa a destring):
 - **generate reg_num = real(reg_str)**
 - Ver diferencias con
encode reg_str, gen(reg_num1)
- Si tengo string con palabras encode
- Si tengo string con numeros (ambas) pero ojo con las diferencias!

Tipo de datos

- Tipo de dato fecha
- Stata trabaja con fechas a través de un formato denominado “elapsed dates” (%d)
- La fecha se representa con un número que surge de considerar la diferencia en cantidad de días respecto al 1ro de enero de 1960
- 0 se corresponde con 1ro de enero de 1960
- -3 con 29 de diciembre de 1959
- Se pueden representar fechas desde el 1/1/100 al 31/12/9999

Tipo de datos

- Tipo de dato fecha
- Funciones para transformar datos con fechas en formato %d
 - date: permite convertir una variable string en %d
 - mdy: permite tomar tres argumentos (día, mes y año) y convertirlos en %d

Tipo de datos

- Ejemplo date:
 - Agregamos los datos de la fecha de nacimiento de algunos individuos (falsos)
 - Con diferentes formatos de fecha
 - Convertir esta información de fechas en string en elapsed date
 - Por qué %d? Es posible manipular mejor los datos (cálculos, orden, etc.)

Tipo de datos

- Ejemplo date:
 - Se genera una variable %d que representa la fecha de la variable string *fecha*
 - generate sdate = date(fecha,"DMY")
 - Ver Resultados
 - Se genera como resultado una variable numérica que almacena la diferencia de días con respecto al 1/1/1960
 - Se observa que la función date puede leer múltiples formatos de fechas (ver Help)

Tipo de datos

- Ejemplo date:
 - Hay una observación que stata no la puede interpretar (missing)
 - Por qué?
 - Se puede solucionar

Tipo de datos

- Ejemplo date:
 - generate sdate1 = date(fecha,"DM19Y")
 - generate sdate2 = date(fecha,"DMY",2030)
 - En el primer caso se indica la centuria o siglo por defecto en el caso de los años definidos con dos números
 - En el segundo caso se indica el año máximo de conversión

Tipo de datos

- Ejemplo date:
 - El formato elapsed date es útil para stata pero ilegible para la mayoría de la gente. Existe la posibilidad de dar formato a la visualización de una fecha
 - `format sdate2 %d`
 - `format sdate2 %dD_m_CY`
 - `format sdate2 %dD/m/Y`
 - `format sdate2 %dD/M/Y`
 - `format sdate2 %dD/n/Y`
 - `format sdate2 %dD/N/Y`

Tipo de datos

- Funciones para extraer información de una variable fecha (sdate1):

`year(sdate1)`

`month(sdate1)`

`day(sdate1)`

`halfyear(sdate1)`

`quarter(sdate1)`

`week(sdate1)`

`dow(sdate1)`

`doy(sdate1)`

Tipo de datos

- Ejemplo date:
 - Otra función para transformar variables en fechas con formato %d es mdy():
 - gen XX = mdy(mes, dia, anio)
 - gen XX = mdy(mes, dia, 1900+anio)
 - Partir de la variable sdate1